

Data-Flow Testing and Coverage Concepts

Week 6 Lecture — SE 413/513: Software Testing

Dr. Kamrul Hasan

Grand Valley State University

Fall 2026

Today's Plan (60 minutes)

Segment	Time
1. Introduction & Objectives	3 min
2. Black-Box vs. White-Box Testing: Where We Are	12 min
3. From Control Flow to Data Flow: Defs and Uses	8 min
4. Why It Matters: Data-Flow Defects & OpenSSL	6 min
5. Def-Clear Paths, DU Pairs & <code>sum_positive()</code>	15 min
6. Data-Flow Coverage Criteria & the Hierarchy	9 min
7. Practice, Pitfalls & Wrap-Up	7 min
Total	60 min

Based on this week's reading, *Data-Flow Testing and Coverage Concepts*. Slides tagged **From earlier weeks** reuse examples from Week 3, Week 4, and Assignment 1. Items left for independent study are listed on the "What We Didn't Cover" slide.

Learning Objectives

By the end of today's class, you should be able to:

- Contrast **black-box** and **white-box** test design, and place data-flow testing within white-box testing.
- Explain how data-flow testing differs from control-flow-based coverage.
- Identify **definitions** and **uses** of a variable, and distinguish a **c-use** from a **p-use**.
- Explain what a **def-clear path** is, and identify the **DU pairs** of a small function — including a **loop-carried** pair.
- Define **All-Defs**, **All-Uses**, and **All-DU-Paths**, and relate them to each other and to statement and branch coverage.
- Design a small test set that achieves a target data-flow criterion.

Part 1

Black-Box vs. White-Box Testing: Where We Are

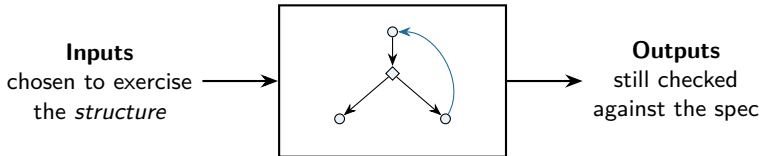
~12 minutes

Two Ways to Look at the Same Code

Black-box (specification-based)



White-box (structure-based, "glass box")



Black-Box vs. White-Box at a Glance

	Black-box	White-box
Tests derived from	Requirements / specification	Source code structure
Need to see the code?	No	Yes
Core question	“Did I pick good, representative inputs for what this <i>should</i> do?”	“Did my tests actually <i>execute</i> (and feed values through) the code I wrote?”
Techniques	Equivalence partitioning, boundary-value analysis	CFGs; statement, branch, path coverage; data-flow coverage
Blind spot	Code no requirement mentions (dead or extra paths)	Missing behavior; exact boundary values
In this course	Week 3, Assignment 1	Week 5 (control flow), Week 6 (data flow)

Black-Box Recap: The Registration Fee

From earlier weeks: Assignment 1 Spec: \$100/credit for the first 12 credits; \$75/credit above 12, up to 18. total credits must be 0–18, otherwise ValueError. *We never look at the code.*



red dots = boundary-value test inputs

```
@pytest.mark.parametrize("credits, fee", [
    (0, 0), (12, 1200),
    (13, 1275), (18, 1650),
])

def test_fee_boundaries(credits, fee):
    assert calculate_registration_fee(credits)
        == fee
```

```
@pytest.mark.parametrize("credits", [-1, 19])
def test_fee_rejects_invalid(credits):
    with pytest.raises(ValueError):
        calculate_registration_fee(credits)
```

Black-Box Recap: What It Is Good At

Every test on the previous slide came from the **spec**, not the code:

- **Equivalence partitioning** said: one representative per class is enough — we don't test all 19 valid values.
- **Boundary-value analysis** said: defects cluster at the edges, so test 0/−1, 12/13, 18/19.
- A wrong `<` vs. `<=` at 12 or 18 is exactly what these tests catch.

The same idea was behind Week 4: `place_order(payment_service)` was tested only through its **interface** — a mocked payment result in, a confirmation message out.

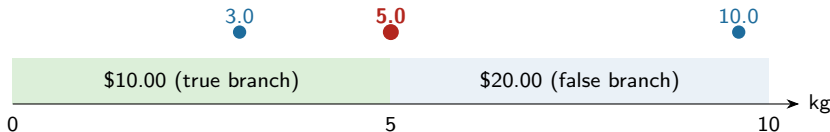
Key Idea

Black-box testing chooses **which values** to try. It cannot tell you whether some line, branch, or variable in the implementation was never exercised at all.

Same Code, Two Lenses: The Shipping-Cost Bug

From earlier weeks: [Week 3](#) & [Week 5](#) **Spec:** packages *up to and including* 5 kg cost \$10.00; heavier packages cost \$20.00. The code has an off-by-one:

```
...  
elif weight_kg < 5:           # BUG: should be <= 5  
    return 10.00  
else:  
    return 20.00
```



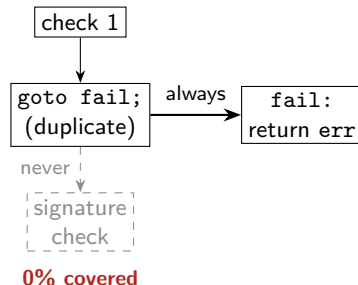
White-box (branch coverage): 3.0 and 10.0 take both branches $\Rightarrow$ **100% branch coverage**, bug *not* found.

Black-box (BVA): the spec's boundary is 5 kg $\Rightarrow$ test 5.0 $\Rightarrow$ expect \$10.00, get \$20.00. **Bug found.**

And White-Box Finds What Black-Box Can't

Last week's "goto fail" case: a duplicated line made the real signature check **unreachable**.

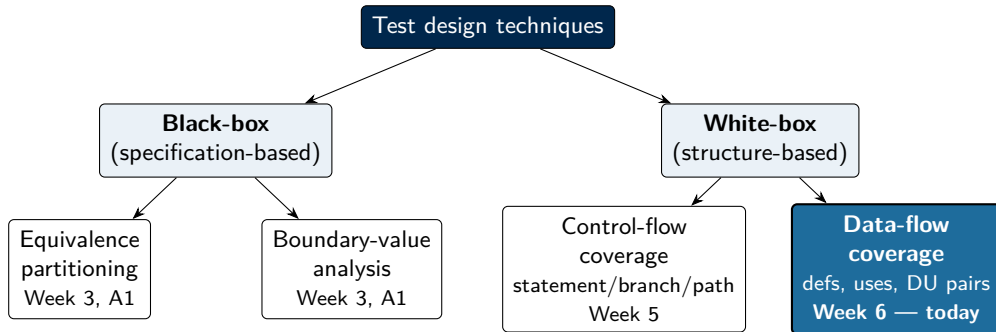
- No requirement says "the signature check must be *reachable*" — black-box tests with valid certificates all passed.
- Any statement-coverage report would show that check at **0% executed**, under every test.



Key Idea

The two families are **complementary**: black-box picks *which values* to test; white-box confirms *nothing in the implementation was left untouched*.

Where Today Fits



Part 2

From Control Flow to Data Flow: Defs and Uses

~8 minutes

Which Code Ran vs. Which Values Arrived

Statement, branch, and path coverage are entirely about **control**: which statements executed, which branch outcomes were taken, which paths were exercised.

But a function can run *every* statement and *every* branch and still compute the wrong answer, because:

- a variable was read before it was properly set,
- a value was overwritten before anything used it, or
- a stale value from an earlier iteration leaked into a later computation.

Key Idea

Control-flow coverage asks *which code ran*. Data-flow coverage asks *whether the right values reached the right places*. They are complementary, not competing.

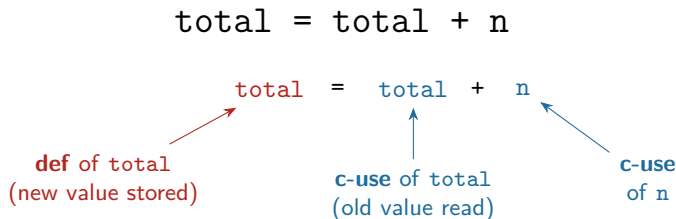
Definitions and Uses

Two events can happen to a variable at a statement:

- **Definition (def)** — the variable is **assigned** a value: assignment, input, loop variable binding, parameter passing.
- **Use** — the variable's current value is **read**. Two kinds:
 - **c-use** (computation-use): the value feeds a computation or output.
 - **p-use** (predicate-use): the value decides which branch is taken.

Event	Meaning	Example
Definition (def)	Variable is assigned a value	<code>total = 0</code>
c-use	Value read for a computation or output	<code>return total</code>
p-use	Value read to choose a branch	<code>if n > 0:</code>

One Statement, Several Events



The old value of `total` is *read*, then a new value is *written* — all in one statement.

Tip

When reading unfamiliar code, tag every statement that touches a variable as **D** (def), **P** (p-use), or **C** (c-use). That is exactly what static data-flow analysis tools do, and what we will draw on a CFG shortly.

Quick Check: Tag the Events

```
def apply_membership_discount(price,  
                               is_member):  
    if is_member:  
        price = price * 0.9  
    return price
```

(From the Week 5 control-flow reading.)

Quick Check — Think, Then Discuss

For price and is_member, mark each statement D, C, or P.

Quick Check: Tag the Events

```
def apply_membership_discount(price,  
                               is_member):  
    if is_member:  
        price = price * 0.9  
    return price
```

(From the Week 5 control-flow reading.)

Quick Check — Think, Then Discuss

For price and is_member, mark each statement D, C, or P.

- `def ... (price, is_member):` **D** both (parameters)
- `if is_member:` **P** is_member
- `price = price * 0.9:` **C** then **D** price
- `return price:` **C** price

Part 3

Why It Matters: Data-Flow Defects

~6 minutes

Three Defect Patterns Control Flow Can't See

Pattern	What goes wrong
Use before def	A variable is read before it was meaningfully assigned on that path — e.g., an error branch referencing a result that was never set. May silently return garbage, a default, or a leftover value.
Dead (unused) def	A value is assigned but never read. Often harmless — or a sign that an intended update elsewhere is missing.
Killed def	A value is computed, then overwritten before anything reads it. If the second assignment is the bug, coverage can be 100% while the wrong value is used.

Key Idea

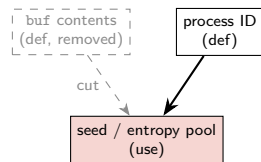
Shared signature: the *code* executed exactly as written (control-flow coverage is satisfied), but the *data* did not flow the way the programmer assumed.

Case Study: Debian OpenSSL Weak Keys (CVE-2008-0166)

In 2006, a Debian maintainer silenced Valgrind warnings about reading *uninitialized memory* in OpenSSL's random-number generator by commenting out:

```
MD_Update(&m, buf, j); /* in ssleay_rand_add(), crypto/rand/md_rand.c */
```

- Those reads were **definitions** feeding the entropy pool — on purpose.
- Remaining randomness: only the **process ID**, 0–32,767.
- Every key made 2006–May 2008 came from $\leq 32,768$ possibilities: SSH, SSL/TLS, OpenVPN, DKIM keys.



only 32,768 possible seeds

Real-World Case Study

Every remaining line ran exactly as written, so statement and branch coverage saw nothing wrong. The defect was entirely about **which values reached the seed**.

Part 4

Def-Clear Paths, DU Pairs & `sum_positive()`

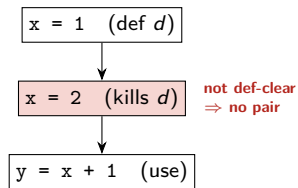
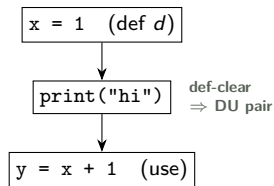
~15 minutes

Def-Clear Paths and DU Pairs

A path from a def of x to a use of x is **def-clear** if **no other def of x** occurs along it. Then the value read at the use is the value written at that def.

A **DU pair** (x, d, u) : a def d of x and a use u of x , connected by **at least one def-clear path**.

DU pairs are what data-flow criteria count — the way branch coverage counts CFG edges.



Common Pitfall: Nearby $\neq$ Paired

Common Pitfall

A def and a later use of the same variable do **not** automatically form a DU pair. If another definition occurs on *every* path between them, the original value never survives to reach that use — it was **killed** first. Only a genuinely def-clear path creates a DU pair.

And the reverse surprise, coming up next: a def and a use can form a pair even when the use appears *earlier* in the source text — if a loop carries the value back around.

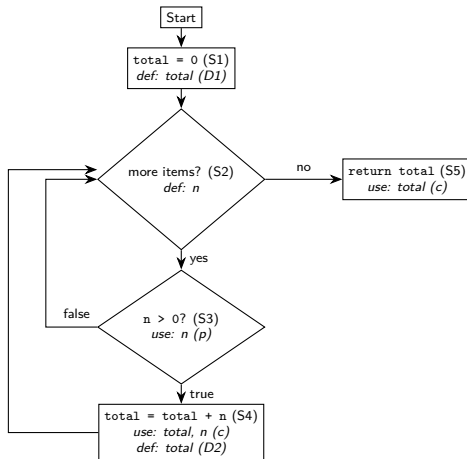
Worked Example: `sum_positive()`

```
def sum_positive(numbers):  
    total = 0                # S1  
    for n in numbers:        # S2  
        if n > 0:            # S3  
            total = total + n # S4  
    return total             # S5
```

Small, but it has: a def before the loop, a def that reads its own old value, a fresh loop variable, a p-use, and a def that can reach a use on a *later* iteration.

Stmt	Def/Use events
S1	def total (D1)
S2	def n (fresh each iteration)
S3	p-use n
S4	c-use total, c-use n; def total (D2)
S5	c-use total

Annotated Control-Flow Graph



Two **back-edges** return to the loop condition:

- from S4, after the sum is updated;
- from S3's *false* outcome, when an element is skipped.

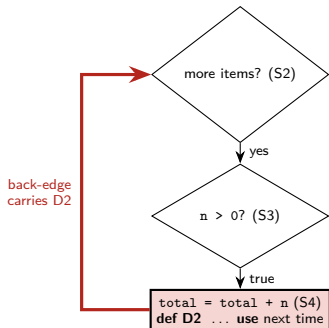
Either one can carry D2's new value of `total` into a *later* pass through S4.

The Seven DU Pairs

#	Var	Def	Use	Notes
1	total	D1 (S1)	S4 (c)	first time through the loop
2	total	D1 (S1)	S5 (c)	def-clear when S4 never runs (empty list, or no element > 0)
3	total	D2 (S4)	S4 (c)	loop-carried: D2 reaches S4 on a <i>later</i> iteration
4	total	D2 (S4)	S5 (c)	S4 runs at least once, then the loop exits
5	n	S2	S3, true (p)	iterations where the element is > 0
6	n	S2	S3, false (p)	iterations where the element is ≤ 0
7	n	S2	S4 (c)	only when the S3 predicate is true

Pair #3 is the genuinely new idea beyond last week's control-flow reading.

Spotlight: The Loop-Carried Pair (#3)



- Def and use are the **same statement**.
- No single pass through the body creates this pair — it exists only because `total` survives *unmodified* into a later iteration.
- It needs at least **two** positive elements to exercise.

Key Idea

A def need not come before its use in the source text: a def at the *end* of a loop body can reach a use at the same statement, or earlier in the body, on the next iteration.

Three Tests Cover All Seven Pairs

Input	Result	New pairs covered
<code>[-1]</code>	0	#2 ($D1 \rightarrow S5$); #6 ($n \rightarrow S3$ false)
<code>[3]</code>	3	#1 ($D1 \rightarrow S4$); #4 ($D2 \rightarrow S5$); #5 ($n \rightarrow S3$ true); #7 ($n \rightarrow S4$)
<code>[3, 5]</code>	8	#3 ($D2 \rightarrow S4$, loop-carried)

Quick Check — Think, Then Discuss

`[-1]` and `[3]` already give **100% branch coverage** of `sum_positive()`. What could still be wrong with the accumulator that only `[3, 5]` would reveal? (Hint: imagine `S4` were `total = 0 + n.`)

Part 5

Data-Flow Coverage Criteria & the Hierarchy

~9 minutes

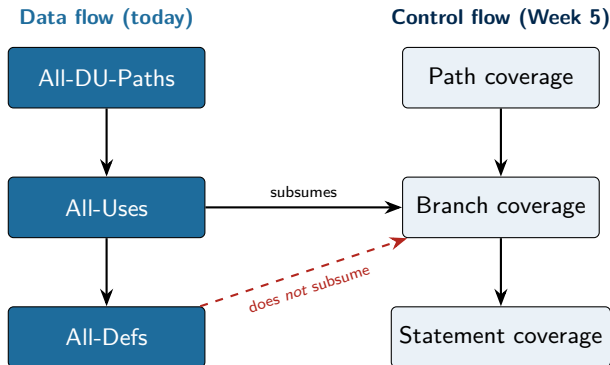
Three Criteria of Increasing Strength (Rapps & Weyuker)

Criterion	Requirement
All-Defs	Every definition reaches <i>at least one</i> use (some use, not necessarily all)
All-Uses	Every definition reaches <i>every</i> use it can reach — every DU pair covered
All-DU-Paths	Every <i>distinct</i> def-clear path for every DU pair is covered, not just one

Applied to `sum_positive()`:

- `[-1], [3]` $\Rightarrow$ **All-Defs** (D1, D2, and n's def each reach a use) — but *not* All-Uses: pair #3 is missing.
- `+ [3, 5]` $\Rightarrow$ **All-Uses** — but *not* All-DU-Paths: pair #3 can also run through a skipped element, `[3, -2, 5]`, and through any number of them.

How the Criteria Relate



Arrow = "100% of the left/upper criterion guarantees 100% of the one it points to." All-Uses subsumes branch coverage because every branch is a **p-use** of some variable.

Why All-Defs Is Not Enough

- All-Defs is satisfied as soon as each definition reaches **one** use — possibly without ever taking both outcomes of a nearby branch.
- In `sum_positive()`, the All-Defs set `[-1], [3]` *happens* to cover both outcomes of S3 — a **coincidence** of this small example, not a guarantee.
- A minimal All-Defs test set can leave whole branches, and loop-carried pairs, completely untested.

Common Pitfall

All-DU-Paths can be infeasible for loops. A loop-carried pair like #3 has one def-clear path per number of intervening iterations — unbounded in general, just like full path coverage. In practice: aim for **All-Uses**, with bounded iteration (zero, one, many).

Data-Flow Testing in Practice

Less common: data-flow *coverage* tools.

- coverage.py, gcov, JaCoCo report statement/branch coverage.
- DU pairs need real static analysis, not just counting executed lines.

Everywhere: def-use *analysis*.

- Compilers, linters, and IDEs: “variable is never used,” “might be uninitialized.”
- Valgrind: runtime analysis — the tool that flagged the OpenSSL reads.

Tip

The practical takeaway: **think in def-use terms** whenever a function has internal state — accumulators, running totals, values updated across loop iterations. That is where data-flow bugs hide.

Part 6

Practice, Pitfalls & Wrap-Up

~7 minutes

In-Class Practice (Reading Exercises 1–2)

```
def first_negative(values):  
    result = None  
    for v in values:  
        if result is None and v < 0:  
            result = v  
    return result
```

Quick Check — Think, Then Discuss

- 1 List every statement that defines or uses `result`; classify each use as c-use or p-use.
- 2 Is there a **loop-carried** DU pair for `result`, like pair #3 in `sum_positive()`?

Work with a neighbor for 3 minutes. Exercises 3–5 are homework.

Common Pitfalls

Common Pitfall

Confusing a DU pair with textual proximity. A def followed later by a use is a pair only if a def-clear path connects them.

Common Pitfall

Assuming All-Defs is enough. It can leave entire DU pairs — including loop-carried ones — untested. It is a minimum, not a thorough criterion.

Common Pitfall

Overlooking loop-carried pairs. Whenever a variable is both defined and used in a loop body, ask: does this def also reach a use on a *later* iteration?

Discussion Question

Quick Check — Think, Then Discuss

A teammate says: “Our function has 100% branch coverage, so there’s no need to think about data flow separately.” Using the subsumption hierarchy, the loop-carried pair in `sum_positive()`, or the OpenSSL case, explain what this claim gets right and what it misses.

Think for a minute, then let’s discuss as a class. (Reading, Exercise 5.)

Key Takeaways

- ❶ **Black-box and white-box testing are complementary:** one picks *which values* from the spec; the other checks what the *implementation* actually exercised.
- ❷ **Data-flow testing targets wrong values reaching a computation** — even when every statement and branch ran.
- ❸ **Every variable touch is a def, a c-use, or a p-use;** one statement can be several at once.
- ❹ **A DU pair requires a def-clear path,** not textual proximity.
- ❺ **Loops create DU pairs that span iterations.**
- ❻ **All-DU-Paths $\Rightarrow$ All-Uses $\Rightarrow$ All-Defs;** All-Uses subsumes branch coverage, All-Defs does not.

What We Didn't Cover Today

In the interest of pacing, these are in the reading but were **not** covered (or only briefly) in lecture — you are responsible for them as reading:

- Section 10, “A Practical Mental Model” — the three think-aloud prompts (also in backup slides).
- Section 11, the full Summary table (also in backup slides).
- Section 12, Exercises 3–5 — **homework** (Exercises 1–2 started in class).
- Section 13, Discussion Questions (backup slides list them all).
- Section 15, References — consult the reading directly for citations.

Also note: the black-box examples today (registration fee, shipping cost, `place_order`) came from Week 3, Week 4, and Assignment 1 — they are not in this week's reading.

Questions?

Homework: Reading Exercises 3–5 (`first_negative()`)

Dr. Kamrul Hasan | Grand Valley State University

Backup Slides

Reference material only — not part of the 60-minute lecture

Backup: A Practical Mental Model (Reading, Section 10)

When you see a variable assigned inside a loop, think:

“Where does this value go next — used again this iteration, overwritten, or surviving to the next iteration? If it survives, that is a DU pair my tests need to exercise deliberately.”

When a report shows 100% branch coverage on a function with an accumulator, think:

“Every decision ran both ways. That says nothing about whether the accumulated value was ever checked after more than one update.”

When reviewing a bug where “the code ran but produced the wrong value,” think:

“Which definition actually reached the point where it went wrong — and was that the one the author intended?”

Backup: Summary Table (Reading, Section 11)

Term	Meaning
Definition (def)	A statement that assigns a value to a variable
Use	A statement that reads a variable's value
c-use / p-use	A use in a computation/output vs. a use that controls a branch
Def-clear path	A path from a def to a use with no intervening redefinition
DU pair	A (variable, def, use) triple connected by at least one def-clear path
All-Defs	Every definition reaches at least one use
All-Uses	Every DU pair is exercised at least once
All-DU-Paths	Every distinct def-clear path for every DU pair is exercised

Backup: Remaining Practice Exercises (Homework)

Using `first_negative()` from the in-class practice slide:

- **Exercise 3:** Design a minimal test set that achieves **All-Uses** coverage on `result`. Show which pair each test covers.
- **Exercise 4:** Find a *smaller* test set that achieves **All-Defs** but not All-Uses. Explain precisely which DU pair it misses.
- **Exercise 5:** “100% branch coverage means no need to think about data flow” — argue what this gets right and what it misses (discussed in class).

Backup: Discussion Questions (Reading, Section 13)

- 1 What is the difference between a c-use and a p-use, and why does it matter for relating data-flow coverage to branch coverage?
- 2 What does it mean for a path to be “def-clear,” and why is it required for a DU pair?
- 3 Why can a statement both use and define the same variable? Give an example other than `total = total + n`.
- 4 Why does pair #3 in `sum_positive()` exist only because of the loop back-edge?
- 5 Why does All-Defs not generally guarantee branch coverage, even though the worked example's All-Defs set happened to?
- 6 Why can All-DU-Paths be infeasible for loops, and how is that similar to full path coverage?
- 7 In the OpenSSL case, which def-use relationship was broken, and why was it invisible to statement and branch coverage?
- 8 Why is data-flow coverage tooling less common in industry than statement/branch tooling?
- 9 Give a plausible bug that use-before-def analysis would catch but 100% statement coverage would not.
- 10 How do data-flow and control-flow criteria complement each other in a real test strategy?

Backup: References (from the Reading)

- Rapps, S., & Weyuker, E. J. (1985). Selecting software test data using data flow information. *IEEE Transactions on Software Engineering*, SE-11(4), 367–375. <https://doi.org/10.1109/TSE.1985.232226>
- ISTQB (2024). *Certified Tester Foundation Level (CTFL) Syllabus, v4.0.1*.
- Debian Security Team (2008). *[DSA 1571-1] New openssl packages fix predictable random number generator*. CVE-2008-0166.
- Hasan, K. (2026). *Software Testing*. Grand Valley State University. <https://github.com/mdkamrulhasan/software-testing>